INT450 Large Language Models & Agentic AI

Industry-Sponsored Capstone Project Briefs

Two contrasting, build-in-parts problems for final-year teams of three, designed to run end-to-end on a laptop with free-tier or local models.

Prepared for	Prof. K. Ramkumar, Associate Dean, Academics, School of Computing, SASTRA Deemed University
Course	INT450 — Large Language Models & Agentic AI (Semester VI, 3-1-0-4)
Prepared by	Vikas Mishra, Platform Architect, Cloud & AI, Google Cloud with Sanjeev Gopinath and Sai Balaji Rajaraman
Format	Teams of 3 · ~2-month duration · 2 problem statements (assign one per team by fit)
Version / date	v1.0 · 24 August 2026

A note on intent

These briefs deliberately go beyond 'build a chatbot.' Each problem hides a failure mode that a single well-written prompt cannot solve, so teams are forced to design retrieval, memory, a knowledge representation, a safety/policy layer, and above all their own evaluation. The integrity model (Section 0.3) is built around one enforceable guarantee: every team curates its own private dataset with planted 'traps,' so there is no shared answer key and no model has seen the data. Students may use AI assistants freely; the deliverables are structured so that judgment, curation, evaluation and defense, the parts AI cannot fake for them.

0. How to use this document

0.1 What's inside, and who does what

This pack contains two independent problem statements: Problem A (TimeMachine) and Problem B (IncidentOps). They are intentionally contrasting: A is knowledge- and retrieval-heavy (RAG, knowledge graphs, temporal reasoning, faithfulness); B is orchestration- and action-heavy (multi-agent systems, tool use, planning, safety policy). Assign one problem per team based on the team's interests. The two teams already mapped can start immediately,one on each problem.

- **Self-contained.** Each problem has its own context, private-dataset mandate, six modules (M0–M5), architecture guidance, rubric, and viva bank.
- **Laptop-first.** Every core deliverable runs offline. Any LLM is allowed (a free hosted tier or a local model via OpenRouter/Ollama/Hugging Face). Vector store and graph run locally. Target spend: ₹0.
- **Framework-agnostic.** Teams choose from the course-covered stacks,Google ADK, OpenAI Agents SDK, CrewAI, LangGraph, or AutoGen and must justify the choice.

0.2 The Core / Stretch model (keeping it doable)

Every module is split into a Core tier and a Stretch tier. Core is what a committed, average team can complete in the time given hitting Core across all modules is a solid, well-earned pass. Stretch is where distinction lives: temporal knowledge-graph completion, PEFT/LoRA fine-tuning of a small helper model, MCP servers, A2A messaging. The hard course outcomes are deliberately placed in Stretch so they never block completion. The intent is that a clear majority of teams finish a genuinely working Core system they understand.

Tier	What it means	Who it's for
Core	A working, honestly-evaluated system that solves the base problem on the team's own data. Every module defines its Core bar explicitly.	Every team. This is the pass-level target.
Stretch	Advanced capability that exercises the harder K3 outcomes (KG completion, PEFT, MCP/A2A, self-correction).	Teams aiming for distinction, once Core is solid.

0.3 Integrity model — why AI cannot do this for the students

We do not try to ban AI assistants; final-year students will use them, and learning to use them well is part of the job. Instead the work is shaped so that AI alone cannot produce a passing submission, and AI use is declared and graded. Five mechanisms do the work:

1. Team-curated private dataset (the backbone). Each team builds its own corpus / scenario suite and plants a documented taxonomy of 'traps' (superseded rules, contradictions, red-herring incidents, escalate-only cases). No two teams share ground truth, so there is nothing to copy and no model has seen the data.
2. Held-out test split. Teams build their own evaluation harness and are scored on a private test split they do not inspect during development. A demo that only looks good on cherry-picked inputs earns little.
3. Decision log. Every non-obvious choice (chunking, retrieval, precedence rules, escalation thresholds, memory design) is recorded with its rationale and the alternatives rejected. This log is graded and is the script for the viva.

4. AI-use appendix. Teams declare which assistant they used, for what, and what they kept, rejected, or had to fix. Honest, reflective use scores higher than an undocumented copy-paste — which is usually obvious anyway.
5. Lightweight defense (recommended, near-zero prep for staff). A 10-minute viva at each milestone using the supplied question bank, plus a glance at git history, confirms the team understands what it submitted.

Practical guarantee for the instructor: because the dataset, its traps, and the held-out split are unique to each team, a solution handed to an LLM in one shot cannot reproduce a team's numbers. The interesting work, deciding what a trap should be, what 'faithful' means, when to refuse, when to escalate, is exactly the work an LLM cannot do on the team's behalf.

0.4 What each team submits

- **Code repository** with a clear README and a one-command way to reproduce the evaluation (a Makefile or a single script).
- **Private dataset** corpus/scenarios + metadata + traps.md ledger + gold set with the dev/held-out split.
- **Evaluation report** dev vs held-out numbers, per-trap breakdown, and an honest error analysis (where it still fails and why).
- **Decision log** and AI-use appendix (templates in the Appendix).
- **10-minute demo** and a short architecture diagram.

Problem A — TimeMachine: A Temporal Policy-Compliance Copilot

Knowledge- and retrieval-centric · center of gravity: RAG, knowledge graphs, temporal reasoning, faithfulness and calibrated refusal.

A.1 Industry context

Every organisation runs on written rules that never stop changing: security standards, HR handbooks, data-retention policies, procurement rules, regulatory circulars. A clause approved in 2021 is amended in 2023 and partly superseded in 2024. So when an employee, an auditor, or an automated system asks 'Is X allowed?', the honest answer depends on when they are asking about and which version governs. Most 'policy chatbots' get this wrong: they retrieve the paragraph that is most similar to the question and answer confidently, even when that paragraph was repealed eighteen months earlier. In a regulated enterprise that is not a cosmetic bug; it causes audit findings, rework, and fines.

The task is to build an assistant that answers a policy question as of a point in time, cites the exact governing clause, surfaces conflicts between overlapping rules, and critically refuses or escalates when the policy is silent, ambiguous, or self-contradictory, instead of inventing an answer.

A.2 The core challenge (why this is not a chatbot)

- **Temporal grounding.** Every rule has an effective date, sometimes a sunset date, sometimes it supersedes an earlier rule. 'What is the WFH policy?' is under-specified; 'What was the WFH policy on 12 March 2023?' has exactly one correct answer — and it may not be the newest document.
- **Conflict and supersession.** Two documents can both look relevant while one overrides the other. The system must reason about precedence (recency, specificity, issuing authority), not just similarity.
- **Faithfulness.** Answers must trace to a specific clause id. An unsupported claim is a failure, not a rounding error.
- **Calibrated refusal.** For ambiguous, missing, or contradictory cases the correct behaviour is to decline with a reason or escalate — the one thing naive systems never do.

A single LLM prompt over concatenated documents fails all four. That gap is what pushes teams into real retrieval design, a temporal knowledge representation, a precedence policy, and an evaluation harness.

A.3 Scope and constraints

- **Offline.** Runs on a laptop. Any LLM (free hosted tier or local via Open Router/Ollama/Hugging Face). Local vector store (FAISS/Chroma). Graph via networkx, SQLite, or rdflib.
- **Framework.** Team's choice among ADK, OpenAI Agents SDK / CrewAI / LangGraph / AutoGen; justify the choice in the decision log.
- **Corpus size.** ~40–80 documents is plenty, this is a reasoning problem, not a scale problem. Quality and temporal structure matter far more than volume. English corpus.

A.4 The private-dataset mandate (integrity backbone)

Each team builds its own policy corpus. Good sources: SASTRA's real public regulations and circulars across several years; a fictional company handbook you author; or a public regulator's dated circulars. Requirements:

- **Documents.** 40–80 documents, each with metadata: doc_id, title, issuing_authority, effective_date, optional sunset_date, supersedes (doc_id), version. The window must span at least four years, with at least three documents that amend or supersede earlier ones.

- **Gold Q&A set.** ≥ 60 questions, each with: question, as_of_date, expected answer, governing doc_id(s) + clause, and a label (answerable / refuse / escalate / conflict). Split 70/30 into dev and a held-out test set that is NOT inspected during development.

Trap taxonomy — plant each of these and record them in a graded traps.md ledger:

ID	Trap	What it tests
T1	Superseded-but-similar: an old clause that is the closest semantic match to a common query but is no longer in force.	Does retrieval blindly follow similarity?
T2	Contradiction: two in-force documents that conflict on the same point.	Conflict detection and precedence.
T3	Temporal edge: a query whose answer flips as the as-of date crosses an effective or sunset boundary.	Temporal reasoning.
T4	Silent/missing: a plausible question the corpus genuinely does not answer.	Calibrated refusal (don't hallucinate).
T5	Ambiguous scope: a rule for 'full-time staff' queried for a contractor.	Clarify rather than over-apply.

Because the corpus and its traps are unique to each team, there is no shareable answer key and no model has seen the data — the work cannot be copied or one-shotted.

A.5 Modules (build in parts)

M0 - Framing & data plan (Week 1)

Goal	Agree the domain, corpus source, and what 'good' means.
Deliverable	A 2-page charter: corpus source, metadata schema, chosen framework (with reasoning), and the precedence rules you will adopt (recency vs specificity vs authority).
Decisions to justify	Corpus source; how you will encode supersession and effective/sunset windows.
Checkpoint	Charter review with the instructor.

M1 - Corpus curation & trap injection (Week 2)

Goal	Build the private corpus, its metadata, the traps, and the gold Q&A.
Deliverable	/corpus (documents + metadata), traps.md, gold_qa.jsonl with the dev/held-out split.
Core	40–80 docs with valid metadata; all five trap types present and documented; ≥ 60 gold questions.
Stretch	Multi-authority corpus (e.g., central policy vs department override) to make precedence genuinely non-trivial.
Checkpoint	Dataset review — is the trap taxonomy actually present and labelled?

M2 - Baseline RAG + evaluation harness (Weeks 3–4)

Goal	A working retrieve-then-answer baseline AND the harness that scores it — built together, so you can measure every later change.
Deliverable	Baseline pipeline + eval.py computing retrieval hit@k, answer faithfulness (does the citation match the governing clause?), and refusal accuracy; baseline scores on the dev set.
Decisions to justify	Chunking strategy; embedding model; k; and your operational definition of 'faithful.'
Core	Hybrid keyword + vector retrieval; answers carry an exact-clause citation; the harness runs end to end.
Stretch	Add a re-ranker and query rewriting; compare and report the delta.
Checkpoint	Demo the baseline and the first eval numbers. Expect it to fail the T1/T3 traps — that failure is the point, and motivates M3–M4.

M3 - Temporal grounding & citations (Weeks 4–5)

Goal	Make answers correct as of a date, and enforce clause-level citations.
Deliverable	A temporal filter/reasoner that restricts retrieval to the documents in force for the query's as_of_date; measured improvement on the T1 and T3 traps.
Decisions to justify	How you represent effective/sunset windows; what to do when as_of is omitted — assume today, or ask the user?
Core	As-of filtering works; T1/T3 accuracy visibly improves over M2.
Stretch	Handle relative time ('last quarter'); answer 'what changed between date X and Y?'
Checkpoint	Demo temporal queries; show the trap-level improvement.

M4 — Knowledge graph, conflict detection & policy-aware refusal (Weeks 6–7)

Goal	Represent rules as a (temporal) knowledge graph; detect conflicts and supersession; refuse or escalate correctly.
Deliverable	A KG (networkx/rdflib/SQLite) linking rules ↔ entities ↔ dates ↔ supersedes; a conflict detector; and a decision gate that returns answer / clarify / refuse / escalate with a reason. Measured on T2/T4/T5.
Decisions to justify	Your precedence-resolution algorithm; refusal criteria/thresholds; how memory carries as_of context across a conversation.
Core	Detect and report conflicts; apply rule-based precedence; refuse on genuinely missing information.
Stretch	Temporal KG completion — infer a missing relation using a translation/rotation/multiplicative embedding (Unit II); PEFT/LoRA a small classifier that routes answerable vs refuse.
Checkpoint	Demo conflict handling and refusal behaviour.

M5 — Final evaluation, report & defense (Week 8)

Goal	Run on the held-out set, write it up, defend it.

Deliverable	Evaluation report (dev vs held-out, per-trap breakdown, honest error analysis), decision log, AI-use appendix, a 10-minute demo, and the repository.
Decisions to justify	Where the system still fails and why — a credible error analysis scores higher than inflated numbers.
Checkpoint	Viva using the Problem A question bank.

A.6 Architecture guidance (interfaces, not answers)

Build these as independently testable units, each with a clear input/output so you can swap internals without breaking the rest:

- Ingestion / metadata parser → Indexer (vector + metadata) → Temporal filter → Retriever → Reasoner/answerer (LLM + tools) → KG & conflict service → Decision gate (answer/clarify/refuse/escalate) → Evaluator.

A workable tool surface for the agent:

- search_corpus(query, as_of_date, k) → passages with doc_id + clause_id
- get_clause(doc_id, clause_id) → full clause text + metadata
- check_conflicts(topic, as_of_date) → conflicting rules + precedence verdict
- escalate(reason) → structured hand-off object

A.7 Why naive approaches fail (share with the team)

- Pure vector search returns the most similar clause, not the one in force — it walks straight into T1.
- Stuffing all documents into one prompt loses effective dates and precedence, and the model averages contradictory rules into a confident wrong answer.
- Without a refusal path, T4 questions are always answered — and always wrong.
- Without clause-level citations, you cannot even measure faithfulness, so you cannot tell improvement from luck.

A.8 Evaluation rubric — Problem A

#	Criterion	Wt	Course outcomes
1	Private dataset & trap design: coverage of all five trap types, valid temporal metadata, quality of the gold set.	15	CO4, CO6
2	Core retrieval pipeline works end to end with clause-level citations.	20	CO2, CO4
3	The hard capability: temporal grounding + conflict detection + calibrated refusal, with measured trap-level gains.	20	CO4, CO5
4	Evaluation harness & honest results on the held-out split.	15	CO3, CO4
5	Decision log: quality of tradeoff reasoning and rejected alternatives.	10	CO3, CO5
6	Responsible-AI: refusal calibration, handling of	5	CO6

	stale/PII-like content, limits stated honestly.		
7	Viva & individual understanding (each member).	10	CO1–CO5
8	AI-use appendix & engineering hygiene (git history, README, reproducibility).	5	CO6